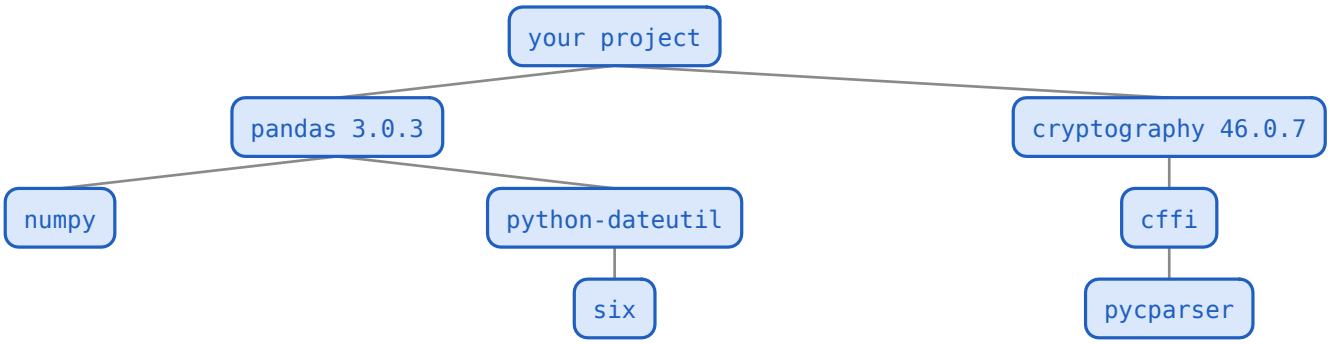
Chapter 19

Choosing pip packages safely

every pip install runs a stranger's code with your permissions

The standard library can't do everything: no friendly HTTP client, no symmetric encryption, no pytest. PyPI has over half a million packages for the rest. Each one you add brings its own dependencies, all written by people you've never met. Here is what two popular packages pull in, read from the metadata on the machine that built this page:




```
1 from importlib.metadata import requires
2 from packaging.requirements import Requirement
3
4 for line in requires("cryptography")[:4]:
5     r = Requirement(line)
6     keep = r.marker is None or r.marker.evaluate({"extra": ""})
7     print(f"{r.name:18} {str(r.specifier):10} {'needed' if keep else 'skipped'}")
```

```
cffi               >=1.14     skipped
cffi               >=2.0.0    needed
typing-extensions  >=4.13.2   skipped
bcrypt             >=3.1.5    skipped
```

line 6: markers: Python version, OS, extras

This chapter gives you a routine for the four decisions every project makes: **whether** to add a package, **which** one, **how** to pin it, and how to notice when the supply chain is attacked.

need	stdlib first	the usual package (and why)
HTTP client	`urllib.request` (Ch 15)	`requests` / `httpx`: sessions, retries
encryption	hashes + HMAC only (Ch 13)	`cryptography`: AES-GCM, Fernet
tests	`unittest` (Ch 17)	`pytest`: `assert`, fixtures, plugins

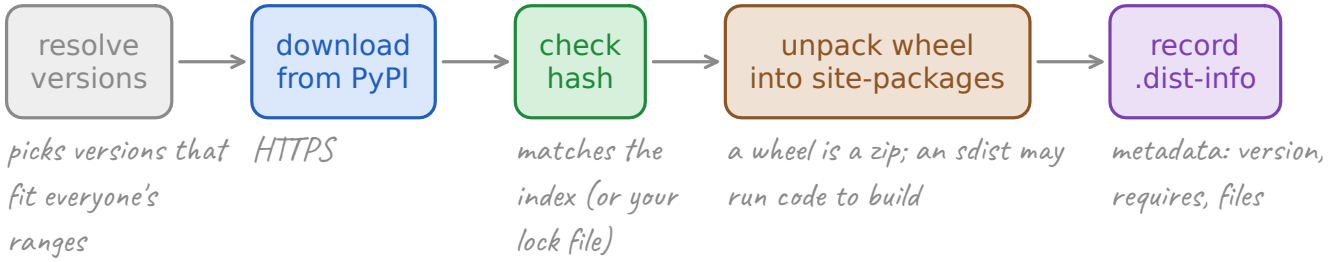
What venv and pip actually do

A **virtual environment** is just a folder with its own `site-packages` and a `python` that uses it, so each project gets its own package versions. Nothing is hidden; look inside one:

a real venv, created while building this page

```
$ python3 -m venv .venv
$ ls .venv
bin
include
lib
lib64
pyvenv.cfg
$ ls .venv/bin
Activate.ps1
activate
activate.csh
activate.fish
python
python3
python3.14
πthon
$ cat .venv/pyvenv.cfg
home = /usr/bin
include-system-site-packages = false
version = 3.14.6
executable = /usr/bin/python3.14
$ .venv/bin/python -c "import sys; print(sys.prefix != sys.base_prefix, sys.prefix)"
True ~/loans/.venv
```

(The `πthon` link is real: a joke alias for version 3.14.)
`pip install requests` inside it then does five things, and each one is a place where trust is needed:




```
1 import sys, sysconfig
2
3 print(sys.prefix == sys.base_prefix)           # True: this book runs outside any venv
4 print(sysconfig.get_paths()["purelib"])       # where pip would put packages
```

```
True
/usr/local/lib/python3.14/site-packages
```

line 3: inside a venv this is False

RULE OF THUMB

One venv per project, never `sudo pip install`, and never install into the system Python (many Linux distributions now refuse: PEP 668, "externally managed environment"). Commit `pyproject.toml` and the lock file, never the `.venv` folder itself (Ch 17's `.gitignore`).

Using a package well: requests

Apply the Ch 7 method to a third-party package exactly as to a stdlib module. A tiny local API to talk to (Ch 15's `http.server`), so this page needs no internet:

```
import json, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class Api(BaseHTTPRequestHandler):
    def do_GET(self):
        found = self.path == "/books/1"
        body = {"title": "Dune"} if found else {"error": "no such book"}
        self.send_response(200 if found else 404)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(body).encode())
    def log_message(self, *args):
        pass

server = ThreadingHTTPServer(("127.0.0.1", 0), Api)
threading.Thread(target=server.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{server.server_port}"
```

stdlib: urllib (Ch 15)

```
import json, urllib.request as ur
from urllib.error import HTTPError
try:
    r = ur.urlopen(f"{base}/books/9", timeout=5)
    print(json.load(r))
except HTTPError as e:                # 404 raises
    print(e.code, json.load(e))
```

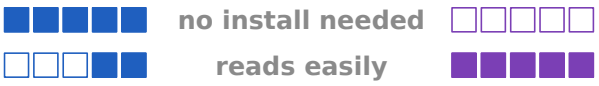
```
404 {'error': 'no such book'}
```

requests

```
import requests

r = requests.get(f"{base}/books/9", timeout=5)
print(r.status_code, r.json())
```

```
404 {'error': 'no such book'}
```



```python
import requests
with requests.Session() as s:                    # keeps connections open between calls
    s.headers["User-Agent"] = "loans/0.3"
    r = s.get(f"{base}/books/1", timeout=5)
    r.raise_for_status()                         # 4xx / 5xx -> requests.HTTPError
    print(r.json(), r.headers["Content-Type"])
    try:
        s.get(f"{base}/books/9", timeout=5).raise_for_status()
    except requests.HTTPError as e:
        print("HTTPError", e.response.status_code)
server.shutdown()
```

```
{'title': 'Dune'} application/json
HTTPError 404
```

line 3: always pass a timeout: the default is to wait forever

LIBRARY CARD **`requests`**

Job	HTTP client: send requests, get `Response` objects. Sync only (`httpx` adds async).
Core objects	`requests.get/post(url, params=, json=, headers=, timeout=)` → `Response`: `.status_code`, `.json()`, `.text`, `.content`, `.headers`, `.raise_for_status()`. `Session()` for many calls to one host.
Lifecycle	request → response; a `Session` is a context manager (`with`) that pools connections.
Errors	`requests.RequestException` ← `ConnectionError`, `Timeout`, `HTTPError` (only if you call `raise_for_status()`).
Traps	no default timeout · a 404 is not an exception · `verify=False` disables TLS checks · `.json()` on a non-JSON body raises.
Docs	requests.readthedocs.io · source: github.com/psf/requests

pytest and cryptography: two more cards

`pytest` runs the same `unittest` suites you already have, and also lets tests be plain functions using plain `assert`. Fixtures replace `setUp`; `parametrize` replaces `subTest` loops:

test_fees.py

```python
import sqlite3
import pytest
from fees import late_fee, DUE, back

@pytest.mark.parametrize("days, fee", [(-3, 0.0), (0, 0.0), (1, 0.25), (20, 5.0), (90, 5.0)])
def test_fee(days, fee):
    assert late_fee(DUE, back(days)) == fee        # failures show both values

@pytest.fixture
def db():
    conn = sqlite3.connect(":memory:")
    yield conn                                      # the test runs here
    conn.close()                                    # cleanup, even after a failure

def test_starts_empty(db):                          # asks for the fixture by name
    assert db.execute("SELECT 1").fetchone() == (1,)
```

```
$ python3 -m pip install pytest
$ python3 -m pytest -q
......                                                                   [100%]
6 passed in 0.04s
```

LIBRARY CARD — **pytest**

Job	Find and run tests (`test_*.py`, `test_*` functions, `unittest` classes too); report clearly.
Core objects	plain `assert` (rewritten to show values) · `@pytest.fixture` · `@pytest.mark.parametrize` · `pytest.raises(E)` · built-ins: `tmp_path`, `monkeypatch`, `capsys`.
Lifecycle	collect → set up fixtures → run → tear down (code after `yield`) → report.
Errors	a failed `assert` = test failure; an exception anywhere else = error; `-x` stops at the first.
Traps	a file not named `test_*.py` is silently skipped · module-scoped fixtures share state · too many plugins make runs mysterious.
Docs	docs.pytest.org · `pytest --fixtures` lists every fixture available

LIBRARY CARD	`cryptography`
Job	Vetted cryptography: Fernet and AES-GCM encryption, signatures, key exchange, X.509 certificates.
Core objects	`fernet.Fernet(key)`: `.encrypt`, `.decrypt` (Ch 13). Lower level: `hazmat.primitives.ciphers.aead.AESGCM`, `hazmat.primitives.asymmetric.ed25519`.
Lifecycle	generate or load a key → make the object → encrypt/sign. Keys live outside the code.
Errors	`InvalidToken` (Fernet), `InvalidTag` (AES-GCM), `InvalidSignature`: tampering or wrong key.
Traps	`hazmat` means hazardous: wrong nonce reuse breaks AES-GCM completely · never invent your own scheme · ships compiled Rust code, so install wheels, not source builds.
Docs	cryptography.io · the 'recipes' layer first, `hazmat` only when a protocol demands it

Is this package worth trusting?

Before `pip install`, spend five minutes on the project page and repository. None of these checks is proof; together they separate a well-run project from a risky one:

check	good sign	warning sign
Does the stdlib already do it?	no, or only painfully	yes: one dependency less to trust
Maintenance	releases this year, issues answered	last release years ago, piles of open PRs
People	several maintainers, an organisation	one anonymous account
Users	widely used, depended on by big projects	brand new, few downloads, name like a famous one
Dependencies	few, well-known	dozens for a small job
Security	advisories handled quickly, a SECURITY.md	no way to report problems
Licence	MIT, BSD, Apache-2.0 (permissive)	none at all (you may not use it), or GPL in closed code
Install	wheels for your platform	only an sdist that runs a build script

Version ranges, pins, hashes, lock files

A dependency line says **which versions** you accept. Wide ranges get bug fixes automatically but can change under you; exact pins are repeatable but go stale. The `packaging` library (what pip itself uses) shows what each specifier accepts:

```
from packaging.specifiers import SpecifierSet
from packaging.version import Version

releases = ["2.28.2", "2.31.0", "2.32.3", "2.32.5", "3.0.0"]
for spec in ["", ">=2.31", "~=2.32.0", ">=2.31,<3", "==2.32.5"]:
    ok = [v for v in releases if Version(v) in SpecifierSet(spec)]
    print(f"{spec or '(any)':10} accepts {ok}")
```

← ~=2.32.0 means >=2.32.0 and ==2.32.*

```
(any)      accepts ['2.28.2', '2.31.0', '2.32.3', '2.32.5', '3.0.0']
>=2.31     accepts ['2.31.0', '2.32.3', '2.32.5', '3.0.0']
~=2.32.0   accepts ['2.32.3', '2.32.5']
>=2.31,<3  accepts ['2.31.0', '2.32.3', '2.32.5']
==2.32.5   accepts ['2.32.5']
```

RULE OF THUMB

Libraries declare ranges in `pyproject.toml` (`requests>=2.31,<3`) so they combine with other libraries. **Applications** also commit a **lock file**: every package, direct and indirect, pinned to an exact version **and hash**, so every install is byte-for-byte the same.

A hash pins the exact file, not just the version number: if anyone replaces the file on the server, the install refuses. The idea in five lines of `hashlib` (Ch 13):

```
import hashlib

wheel = b"PK\x03\x04 ... the real wheel's bytes ..."
pinned = hashlib.sha256(wheel).hexdigest()           # recorded in the lock file
for blob in (wheel, wheel.replace(b"real", b"evil")):
    print("install" if hashlib.sha256(blob).hexdigest() == pinned else "REFUSE: hash mismatch")
```

```
install
REFUSE: hash mismatch
```

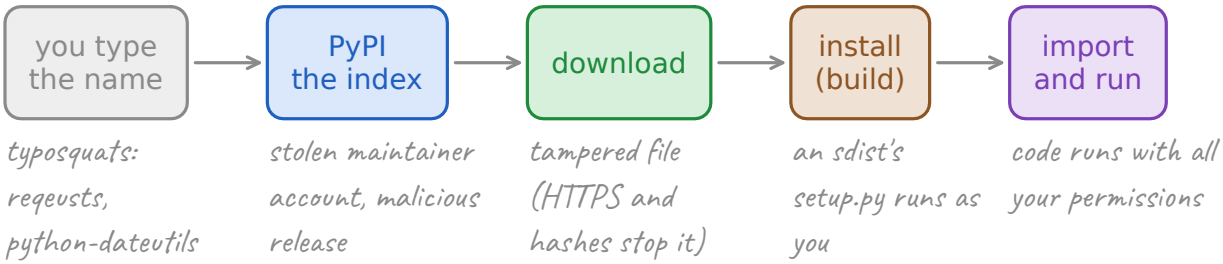
what a hashed lock file looks like (hashes illustrative)

```
$ cat requirements.txt
requests==2.32.5 \
    --hash=sha256:2462f94637a34fd532264295e186976db0f5d453d1cdd31473c85a6a161affb6
urllib3==2.6.3 \
    --hash=sha256:bf272323e553dfb2e87d9bfd225ca7b0f467b919d7bbd355436d3fd37cb0acd4
$ python3 -m pip install --require-hashes -r requirements.txt
```

tool	writes	notes
`pip-tools` (`pip-compile`)	`requirements.txt` with pins and hashes	small, classic, plain pip underneath
`uv`	`uv.lock`	very fast; manages venvs and Python versions too
Poetry / PDM / Hatch	their own lock files	whole project managers
the standard: PEP 751	`pylock.toml`	one lock format tools are converging on

Supply-chain attacks, and the checks that catch them

Attackers aim at the path between the author and your program. Every stage of an install has a known attack:




```
1  import difflib
2
3  popular = ["requests", "numpy", "pandas", "cryptography", "urllib3", "flask", "pytest"]
4  for name in ["reqeusts", "nunpy", "cryptograpy", "requests", "leftpad"]:
5      near = difflib.get_close_matches(name, popular, n=1, cutoff=0.8)
6      if name in popular:
7          verdict = "ok"
8      elif near:
9          verdict = f"did you mean {near[0]}?"
10     else:
11         verdict = "unknown: check it"
12     print(f"{name:12} {verdict}")
```

```
reqeusts     did you mean requests?
nunpy        did you mean numpy?
cryptograpy  did you mean cryptography?
requests     ok
leftpad      unknown: check it
```

line 5: a cheap typosquat alarm (practice 19.5)

a 2024 lock file checked against the PyPI advisory database (OSV)

```
$ python3 -m pip install pip-audit
$ pip-audit -r old-requirements.txt
Found 1 known vulnerability in 1 package
Name    Version ID                  Fix Versions
------- ------- ------------------ ------------
urllib3 2.2.1   GHSA-34jh-p97f-mpxf 1.26.19,2.2.2
```

defence	stops
copy names from the official docs; check the PyPI page	typosquats, look-alike names
lock file with hashes; `--require-hashes`	swapped files, surprise new versions
`pip install --only-binary :all:` where possible	build scripts running at install time
`pip-audit` in CI, then update	known vulnerabilities (only the known ones)
few dependencies; review new ones in code review	everything above, by shrinking the surface
prefer releases with trusted publishing / attestations	uploads from a stolen password

HONEST NOTE

No tool proves a package is safe. Audits only know about **reported** problems, and a clean-looking package can turn malicious in its next release. The strongest defence is the boring one: depend on fewer things, and know why each one is there.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 19.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 19.1` shows other valid solutions and their trade-offs.

`19.1` build from a spec Read a requirement line
`19.2` fix the bug Does this version satisfy the specifier?
`19.3` build from a spec Check downloads against a hashed lock file
`19.4` learn an unseen module Explore importlib.metadata: what is installed, and what does it need?
`19.5` fix the bug A typosquat alarm that knows its names
`19.6` mini-project Mini pip-audit

PASSPORT STAMP ask the same question of every new language (Appendix B)

9. Modules and build

`venv` per project; `pip` resolves, downloads, checks hashes and unpacks wheels into `site-packages`; ranges in `pyproject.toml`, exact pins and hashes in a lock file.

12. Stdlib and tooling

Stdlib: `venv`, `importlib.metadata`, `hashlib`, `difflib`. Third-party staples: `requests`, `pytest`, `cryptography`, `packaging`, `pip-audit`.

IN OTHER LANGUAGES

Every ecosystem has the same parts. JavaScript: npm, `package.json` + `package-lock.json`, and the most famous typosquats. Rust: crates.io, `Cargo.toml` + `Cargo.lock`, `cargo audit`. Java: Maven Central, `pom.xml`. Go: `go.mod` + `go.sum` (hashes built in). C/C++ often vendor code by hand, which is a supply chain too.

DEEP DIVE (your vault)

Vault notes: 19.14 Dependency and supply-chain risk · 19.28 Vulnerability management · 19.17 Cryptographic hash functions · 19.4 Trust and least privilege · 16.28 Reproducible computational environments · 12.5 pytest fundamentals · 12.6 Fixtures and lifetime scopes · 12.21 Continuous integration.